

UPI Offline Mesh Network Simulator

Staff Software Engineer Technical Defense & Interview Masterclass Handbook

Author: Lead System Architect

Target Roles: Senior / Staff Software Engineer, Distributed Systems Engineer

Core Stack: Spring Boot 3.3, Hybrid RSA-2048/AES-256-GCM, Concurrent Idempotency Engine, HTML5
Canvas

Document Version: 2.0 (Exhaustive Master Edition)

Date: July 2026

SECTION 1 — PROJECT OVERVIEW

1.1 Problem Statement & Industry Context

Traditional instant digital payment rails (e.g., UPI, Pix, FedNow) depend strictly on synchronous, 2-phase online clearance across a chain of central authorities: Client $\rightarrow$ PSP $\rightarrow$ NPCI Central Switch $\rightarrow$ Remitter Bank CBS $\rightarrow$ Beneficiary Bank CBS. When cellular coverage drops—such as in underground subways, dense crowd events, high-altitude rural regions, or during power grid failures—all digital commerce grinds to a halt.

1.2 Solution Architecture Overview

The **UPI Offline Mesh Network Simulator** decouples offline payment generation from online ledger settlement. Payments are cryptographically generated, signed, and sealed inside an encrypted payload on the sender's phone. They hop asynchronously across local peer-to-peer (P2P) Bluetooth Low Energy (BLE) / Wi-Fi Direct devices using an epidemic gossip routing protocol until any device in the mesh reaches internet coverage (a "Bridge Node") and flushes the payload queue to the central settlement gateway.

Key Architectural Innovation: The mesh devices act as zero-trust encrypted relays. They route transaction packets without decrypting payload contents or gaining access to sensitive financial parameters.

SECTION 2 — ARCHITECTURAL THOUGHT PROCESS & TECH STACK RATIONALE

Technology Choice	Alternative Rejected	Architectural Justification & Trade-off
Spring Boot 3.3 (Java 17/21)	Node.js / Express	Java provides robust native multi-threading, strict JVM memory layout control, <code>java.util.concurrent`</code> primitives, and Java Cryptography Architecture (JCA) primitives out-of-the-box.
Hybrid Encryption (RSA-OAEP + AES-GCM)	Asymmetric Only (RSA)	RSA-2048 can only encrypt payloads smaller than 245 bytes. Payment instructions containing nonces, signatures, and timestamps exceed this limit. AES-256-GCM provides fast authenticated encryption for payloads of arbitrary size.
ConcurrentHashMap Idempotency	Database Lookup (<code>`SELECT WHERE hash`</code>)	DB lookups introduce disk I/O and transaction lock overhead. <code>`ConcurrentHashMap.putIfAbsent`</code> acts as an in-memory fast-path filter executing in $O(1)$ sub-microsecond time.
Optimistic Locking (<code>`@Version`</code>)	Pessimistic Locking (<code>`SELECT FOR UPDATE`</code>)	Pessimistic locking holds database row locks across network roundtrips, causing thread pool starvation under high concurrent ingestion. Optimistic locking avoids lock contention and relies on version checking during commit.

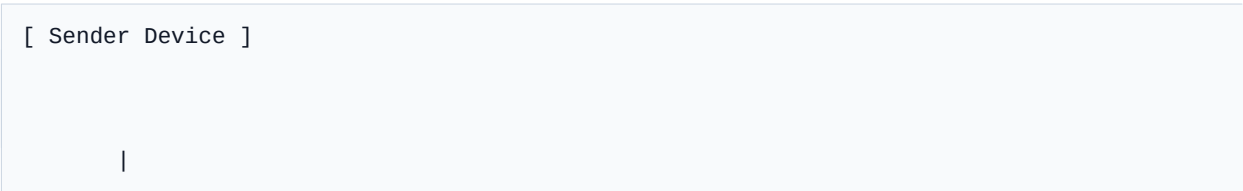
SECTION 3 — SYSTEM DESIGN & CRYPTOGRAPHIC PIPELINE

3.1 Cryptographic Wire Format

The over-the-wire payload structure hopping through intermediate mesh nodes is formatted as:

```
Wire Format = Base64( [RSA-2048 Encrypted AES Key (256 Bytes)] || [AES-GCM IV (12 Bytes)] || [AES-256 Ciphertext + Tag (N + 16 Bytes)] )
```

3.2 Complete Ingestion & Settlement Pipeline



1. Generate PaymentInstruction JSON { sender, receiver, amount, nonce, signedAt }

2. AES-256-GCM Encrypt Payload -> Ciphertext

3. RSA-2048-OAEP Encrypt AES Key -> EncryptedKey

4. Package into MeshPacket { packetId, ttl, ciphertext }

|

(Local BLE Hops over Peer Mesh)

|

[Bridge Node (Enters 4G Range)]

|

5. POST /api/bridge/ingest (Header: X-Bridge-Node-Id, X-Hop-Count)

v

[Server: BridgeIngestionService]

|

6. Compute SHA-256(ciphertext) -> packetHash

7. Idempotency Check: idempotency.claim(packetHash)

└─ [False] -> Return "DUPLICATE_DROPPED" (Fast Exit)

└─ [True] -> Proceed

8. Decrypt Envelope: RSA Decrypt Key -> AES Decrypt Ciphertext

└─ [Fail] -> Return "INVALID" (Tampered)

9. Freshness Check: (Now - signedAt) <= 86400s

└─ [Stale] -> Return "INVALID" (Replay Attack)

10. SettlementService.settle() [@Transactional]

└─ Debit Sender Balance

└─ Credit Receiver Balance

└─ Insert Transaction Record (packetHash UNIQUE INDEX)

SECTION 4 — CODEBASE WALKTHROUGH

- `ServerKeyHolder.java`: Manages the 2048-bit RSA keypair initialized via `@PostConstruct`. Exposes public key for client encryption.
- `HybridCryptoService.java`: Handles AES-256-GCM symmetric payload encryption and RSA-OAEP asymmetric key wrapping. Computes deterministic SHA-256 hashes of ciphertexts.
- `IdempotencyService.java`: Implements thread-safe in-memory duplicate detection using `ConcurrentHashMap.putIfAbsent()`. Runs background eviction via `@Scheduled`.
- `SettlementService.java`: Executes database updates inside an isolated `@Transactional` boundary. Updates balances and creates immutable audit logs.
- `MeshSimulatorService.java`: Simulates 2D coordinate spaces, transmission ranges (\$dist \le range\$), and periodic gossip rounds.

SECTION 5 — EXECUTION FLOW & CONCURRENCY PROOF

The central concurrency challenge occurs during a **"Duplicate Delivery Storm"** when multiple bridge nodes simultaneously upload copies of the exact same offline transaction.



SECTION 6 — DATABASE MASTERCLASS

6.1 Schema Definition & Constraints

```
CREATE TABLE accounts (  
  
    vpa VARCHAR(255) PRIMARY KEY,  
  
    holder_name VARCHAR(255) NOT NULL,  
  
    balance DECIMAL(19, 2) NOT NULL,  
  
    version BIGINT DEFAULT 0  
  
);
```

```
CREATE TABLE transactions (  
  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
    packet_hash VARCHAR(64) NOT NULL UNIQUE,  
  
    sender_vpa VARCHAR(255) NOT NULL,  
  
    receiver_vpa VARCHAR(255) NOT NULL,  
  
    amount DECIMAL(19, 2) NOT NULL,  
  
    signed_at TIMESTAMP NOT NULL,  
  
    settled_at TIMESTAMP NOT NULL,
```

```
bridge_node_id VARCHAR(255) NOT NULL,  
  
hop_count INT NOT NULL,  
  
status VARCHAR(20) NOT NULL,  
  
CONSTRAINT idx_packet_hash UNIQUE (packet_hash)  
  
);
```

SECTION 7 — FRONTEND & CANVAS ANIMATION ENGINE

The user interface features a vanilla JavaScript HTML5 Canvas renderer executing a procedural animation loop (`requestAnimationFrame`). Nodes represent mobile devices with dynamic coordinates (x, y) and coverage ranges R . Connections are calculated dynamically per frame:

```
Distance = Math.sqrt((x2 - x1)^2 + (y2 - y1)^2);  
  
isConnected = Distance <= R;
```

When a gossip round triggers, particle objects step along connection vectors, rendering hop transitions in real time.

SECTION 8 — BRUTAL INTERVIEW CROSS-QUESTIONING & DEFENSES

Q1: Why did you use SHA-256 of the ciphertext as the idempotency key instead of packetId?

Defense: Intermediate mesh devices are untrusted peers. A malicious peer could modify outer JSON fields like `packetId` while preserving the inner encrypted ciphertext payload. Hashing `packetId` would allow duplicate payments to bypass idempotency filtering. Hashing the Base64 ciphertext guarantees that idempotency is mathematically bound to the authenticated payload content.

Q2: How does your system prevent Replay Attacks if a malicious bridge node re-submits a valid transaction weeks later?

Defense: Replay protection is enforced via a two-layer defense. First, `PaymentInstruction` contains an

immutable ``signedAt`` epoch timestamp sealed inside the encrypted payload. During ingestion, the server calculates ``age = Now - signedAt`` and rejects packets older than 86,400 seconds (24 hours). Second, the idempotency cache retains the packet hash throughout the freshness window, dropping duplicate submissions instantly.

Q3: How do you handle double-spending if an offline sender creates two conflicting payments totaling more than their balance?

Defense: The architecture uses a Trust-on-First-Sight (TOFS) model. Whichever bridge node reaches 4G coverage first will have its transaction settled. When the second transaction reaches the gateway, ``SettlementService.settle()`` detects ``sender.balance < amount`` and marks the transaction status as ``REJECTED`` in the database without modifying balances.

SECTION 9 — STAFF ENGINEER ENTERPRISE ROADMAP

1. **Redis Cluster Integration:** Replace JVM ``ConcurrentHashMap`` with Redis ``SETNX`` + TTL keys to scale idempotency horizontally across container nodes.
2. **Hardware Security Module (HSM):** Delegate RSA private key decryption to AWS KMS or Hardware Security Modules (PKCS#11).
3. **Android Keystore Binding:** Enforce client-side RSA key generation inside Android Secure Enclaves so private keys cannot be extracted even from rooted smartphones.
4. **Kafka Ingestion Stream:** Decouple ingestion from settlement using Apache Kafka (``mesh-ingestion-topic``) to buffer high-concurrency bridge flushes.

SECTION 10 — QUICK REFERENCE CHEAT SHEET

Component	Implementation Details
Cryptographic Suite	RSA-2048 (OAEP-SHA256) + AES-256-GCM + SHA-256 Digest
Concurrency Guard	<code>`ConcurrentHashMap.putIfAbsent`</code> + Hibernate <code>`@Version`</code> Optimistic Locking
Replay Guard	<code>`signedAt`</code> Timestamp Sliding Window ($\leq 86400\text{s}$)
Routing Strategy	Epidemic Gossip with TTL Decrement ($\text{dist} \leq \text{range}$)